Day 7 (Morning Session)

Third-Party Modules, npm, package.json, Local vs Global Packages, and ES import/export

Lecture notes - Instructor: Abhishek Purohit - Date: 27 Oct 2026 - Notes for: Sumeet Sharma

#	Topic covered in this lecture
0	Quick carry-over: why a variable named *name* behaves differently in the browser (window.name)
1	Three types of modules: predefined, user-defined and third-party
2	npm (Node Package Manager): install and uninstall commands, installing express
3	What an installation creates: node_modules, package.json, package-lock.json
4	package.json: purpose, contents, creating it with npm init, the entry point (index.js)
5	The dependencies key and why it matters; the role of node_modules; moving a project with npm install
6	package-lock.json
7	Local vs global third-party modules: dependency, -g flag, tools like nodemon
8	ECMAScript import statement vs CommonJS require; "type": "module"
9	export keyword and import of user-defined modules (sum and product)
10	Revision: summary, definitions, differences, mistakes, interview Q&A, exercises, checklist

How to read these notes
Blue boxes hold the instructor's explanation (converted from spoken to written English). Yellow boxes are additional explanations added by me; they were **not** said in the lecture. Red boxes list common mistakes. This lecture did not cover the DOM, events or forms, so those are not part of these notes.

0. Carry-over: the window.name surprise

The recording begins with a browser experiment from the previous topic. A variable called `name` was given the value *abc* and logged; the browser printed *abc*. When the variable was given a different name such as `xyz` and the code still logged `name`, the result was **undefined**.

Instructor's explanation
name is a **predefined property of the window object** (`window.name`). It sets or returns the name of the window's browsing context. Because this property is already declared on window, a variable called *name* at the top level always gets a value due to hosting. Any other variable name that is not a window property will give *undefined* if it has not been assigned. So this is a special case for the name *name*.

Additional note (not from the lecture)
This is a quirk of running code in the **browser**. The same code run on Node.js has no window object, so it behaves differently.

1. Types of Modules: Now Three

In the last session we learned two types: predefined and user-defined. There is one more type, the **third-party module**, so there are three in total.

Type	Who provides it	Installation	Examples
Predefined	Node.js itself (official, out of the box)	Nothing to do; comes with Node.js	fs, http, os, path
User-defined	The developer of the project	Created by the developer	mymodule.js
Third-party	Another organisation (not official Node.js, not you)	Must be **installed manually**	express, cors, axios, bcrypt

Instructor's explanation
A third-party module is not an official predefined module of Node.js and the developer did not create it either. It is **developed and maintained by a third-party organisation**. Think of Node.js as the first party and the developer as the second party; the other organisation is the third party. For example, React is developed by one organisation and Angular by another; they are not part of official Node.js.

Why are third-party modules not installed with Node.js?

The instructor asked the class; the answers were that they are downloaded from another organisation, so the system cannot know them in advance. There are thousands of third-party modules, and Node.js only ships with a limited set of predefined modules. So the **developer has to install them manually**.

Instructor's explanation
In real-time applications, whether back end or front end, we use **a large number of third-party modules** compared with the predefined ones. Predefined modules give limited support, and the functionality we need is often not available there. A real application can easily end up with fifty or even hundreds of third-party modules.

2. npm: Node Package Manager

Instructor's explanation
To install third-party modules we use a **CLI tool called npm**; the full form is **Node Package Manager**. With npm you can install a third-party module, and if it is no longer needed you can uninstall it. npm is a command-line tool, not something like a Play Store with a user interface where you search, click and install. You run it in the command prompt or terminal, just as you run node.

You do **not** need to install npm separately: it is installed automatically when you install Node.js.

Other uses of npm

Installing and uninstalling modules is only one use. The instructor mentioned that npm can also generate the build of your application, run tests, run custom scripts, and even run a Jenkins pipeline. You will learn these one by one later.

2.1 Commands

Command	Meaning
`npm i module-name`	Install a third-party module (short form)
`npm install module-name`	Install a third-party module (long form)
`npm uninstall module-name`	Remove an installed module that is of no further use

The name after the command can be a module name or a package name (both terms are used).

2.2 Example: installing express

Good practice: whenever you create a Node application, create a **separate folder** for it. In class the folder was named my-app (the instructor said this would be an example only; Express as a framework is not being learnt yet, it is just a random example of a third-party module).

1. Create a folder for the application and open a terminal inside it.
2. Run: `npm i express` (or `npm install express`).
3. Wait for the installation. npm reports how many packages were added (in class it said **added 65 packages**).
4. Other examples that could be installed the same way: React, Angular, axios, a MySQL client, and thousands more.

```
npm i express
npm install express       # same thing, long form
npm uninstall express     # remove it again
```

2.3 What gets created after installing a third-party module

> **Instructor's explanation**
> After installing the first third-party module, you will always see **one folder and two files** in your application: the **node_modules** folder, the **package.json** file and the **package-lock.json** file. This happens in every Node application. When you install more modules later, these are not created again; the existing files and folder are just **updated**.

Item	Type	One-line role (details in later sections)
node_modules	Folder	Holds the downloaded JavaScript source code of every installed third-party module
package.json	File	Configuration of the project, including the list of dependencies
package-lock.json	File	Internal mapping between dependencies and their files/folders in node_modules

3. The package.json File

3.1 Why is it so important?

Instructor's explanation
package.json is a very important file. **Every Node application must always have a package.json file**, whether it is a back-end or a front-end application, and whether or not you use any third-party module. It is mandatory and it is good practice to create it at the very beginning. Without it you miss out on most of the important things.

3.2 What does it contain?

Instructor's explanation
The extension .json tells you the content is purely a **JSON object**: key-value pairs only, with no logic. package.json holds the **configuration related to your project**, such as the project name, version, the Git repository, the author, the description, the licences, scripts to test, build, deploy or check code quality, and the main file. The idea is that any other developer can read the file and get this information, so nobody has to ask the project manager every time for something like the Git repository.

Key	What it records
`name`	Name of the project (application)
`version`	Current version of the project (software gets new versions regularly, so keep this updated)
`description`	Short description of the project
`main`	The **main file / starting point** of the application
`scripts`	Scripts to test, build, deploy or check code quality
`repository`	Git repository of the project
`author`	Who created the project
`license`	Licence used
`type`	Which module convention to use: commonjs (default) or module (see section 7)
`dependencies`	List of installed third-party modules and their versions (see section 4)

These are only examples. With time, when you work with React or an Express back end, more properties and configurations will be added.

Why record the main file?

A Node application can have many .js files (file1.js, file2.js, file3.js, even hundreds). A new team member needs to know the **starting point** of the application: the file where the server is created and started. Setting the main file in package.json lets other developers know that by just reading the file.

3.3 Creating package.json: npm init

You can create the file manually (right-click, New File, name it package.json), but ideally you should **not** do this by hand. Use the npm tool instead:

```
npm init
```

Run this inside your project folder before writing any code. It walks you through creating the file, asks for the values, and then generates package.json for you. It only covers the most common items and tries to guess sensible defaults; more can be added manually later.

Prompt	What to enter	Default / instructor's guidance
package name	Name of your project	Defaults to the folder name; press Enter to accept or type your own (my-app was used)
version	Project version	1.0.0 by default; press Enter or give your own (for example 0.0.1)
description	Short text	For example: my first node app
entry point	Main file name	Ideally **index.js**, the file that starts the server
test command	Command to run tests	Skipped for now; testing not covered yet
git repository	Repository URL	Skipped, no repository existed yet
keywords	Optional keywords	Optional
author	Name or e-mail	A random value was used in class
license	Licence	Press Enter for default if not needed
type (convention)	commonjs or module	Default is **CommonJS**; can be changed to ECMAScript later
Is this OK?	Confirmation	Shows the JSON object that will be written; type yes to confirm

Instructor's explanation
The entry-point file should ideally be named **index.js**. This is a recommendation, not a strict rule: you could call it server.js, main.js or app.js, but a relevant name is needed. In every Node application people follow this convention, so if you open any Node project and find index.js, treat it as the starting point where the entire execution begins. It is also described as the welcome file or the file that bootstraps the application. It is just a good practice so that everyone stays on the same page.

After confirming, npm creates package.json. Opening it in VS Code shows keys such as name, version, description, license, author, type, main and scripts. Illustration (values will differ for your project):

```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "my first node app",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "admin",
  "license": "ISC",
  "type": "commonjs"
}
```

The index.js entry point

Once package.json exists you can create your entry point and write code. In class, the instructor created `index.js` with the same http server as in the previous session:

```
// index.js  (CommonJS style)
const http = require("http");

const server = http.createServer((request, response) => {
    response.write("Hello World");
    response.end();
});

server.listen(5000);
```

Additional note (not from the lecture)
The default `scripts.test` line shown above is what npm usually generates; the instructor did not discuss it. The exact license default can also differ by npm version.

4. Why Third-Party Modules Must Be Listed in package.json

4.1 Reason 1: the whole team stays informed

Instructor's explanation
Suppose developer d1 installs five to ten third-party modules, developer d2 installs a few more, and the team has seven or eight developers. Each works on a different feature (one on login, another on something else) and may install new modules for their own feature. How does everybody know which third-party modules the project uses? Not through WhatsApp or e-mail. Every developer **lists the modules in package.json**; a new developer d3 just opens the file and sees the full list with versions. Everyone is on the same page.

4.2 The dependencies key

When you install a third-party module with npm, **npm automatically adds an entry** to package.json under the key **dependencies**, with the module name and the version installed. In class, after installing express, the dependencies key appeared in package.json; after installing cors (another third-party module), a second entry was added.

```
npm i express
npm i cors

// package.json (relevant part; the version numbers on your machine will differ)
{
  "dependencies": {
    "cors": "^x.y.z",
    "express": "^x.y.z"
  }
}
```

Common mistake
Every installed third-party module must have its record in the `dependencies` key of package.json. If an entry is missing, you will get errors when the project is run on another system (explained in 4.4).

4.3 Reason 2: understanding the node_modules folder

In class, the instructor had not written a single line of code, yet the project folder was already about 2 to 3 MB. The properties of node_modules showed roughly **2.17 MB with about 593 files and 129 folders** (the instructor also said roughly 600 files and 130 folders). That is for just two small modules, express and cors.

Instructor's explanation
The **node_modules** folder actually contains the **JavaScript source code downloaded** for each third-party module. When you run npm install express, the files related to express are downloaded and stored there; the same happens for cors and every other module. package.json only keeps the record of which module and which version you installed; the code itself lives in node_modules.

Each module contains many JS files with predefined functions. With fifty or hundreds of modules, node_modules in a real application can be **300 MB, 500 MB, even 600 to 700 MB**, with thousands of folders and lakhs of files.

4.4 The transfer problem and npm install

Instructor's explanation
Imagine you want to give the project to someone, upload it to Git, or move it to another folder or system. Is it practical to move node_modules? **No**, because of its size and its huge number of files and folders. You could tell the other person to run npm i express, npm i cors and so on, but that is not practical if there are 50 modules. Instead we move the **entire source code except node_modules**.

On the new system (which has the source code and package.json but not node_modules) open a terminal inside the project and run:

```
npm install
# or the short form
npm i
```

Instructor's explanation
With **no package name** given, npm reads the package.json file, looks at the **dependencies key**, and installs all the dependencies mentioned there, which recreates node_modules. This is why the installed package must be listed in package.json: if an entry is missing, the module will not be installed on the other system and you will get errors.

Related remarks from class:

- If you delete node_modules and run npm install again, the folder is created again based on the dependencies.
- Deleting or copying a huge node_modules folder (500 to 600 MB, lakhs of files) takes a very long time, maybe 15 to 30 minutes, and on a system with less RAM or a weaker CPU it can even go into a not responding state. A single 1-2 GB file would delete immediately; it is the number of files that hurts.
- The instructor stressed that these points can be asked in interviews.

Additional note (not from the lecture)
A common practice (not mentioned in this lecture) is to list `node_modules` in a `.gitignore` file so it is not pushed to Git.

Situation	Command	What happens
Install one new module	`npm i package-name`	Downloads it into node_modules and adds it to dependencies
Project copied without node_modules	`npm i` / `npm install`	Reads dependencies in package.json and installs all of them
Remove a module	`npm uninstall package-name`	Removes the module

5. The package-lock.json File

Instructor's explanation
As the name suggests, this is a **lock file**. It is generated automatically and you are **not required to change anything** in it, and you should not touch it. It is for the internal purpose of your Node application. package.json lists your dependencies; package-lock.json keeps the **mapping**: for each dependency, which folders and files in node_modules belong to it, along with related details and configurations (in class entries such as a body-parser path inside node_modules were visible). It is used internally whenever you run the application.

If you install another package later (for example `npm i bcrypt`), you do nothing manually: package.json is updated, the files go into node_modules and the mapping is updated in the lock file.

Instructor's explanation
Every Node application in the world looks like this, back end or front end: a **package.json**, a **package-lock.json**, an **entry point** (index.js) and a **node_modules** folder.

File / Folder	Edit it by hand?	Moved to another system / Git?
package.json	Yes, it is your configuration (but let npm manage dependencies)	Yes
package-lock.json	No, never; auto-generated	Part of the project (it is not the bulky part)
node_modules	No; managed by npm	No, too bulky; recreate it with npm install

File / Folder	Edit it by hand?	Moved to another system / Git?
index.js (entry point)	Yes, your code	Yes

Additional note (not from the lecture)
The instructor did not say whether package-lock.json should be copied. Normally it is kept with the source code, which is my addition.

6. Local vs Global Third-Party Modules

There are three top-level types of modules: predefined, user-defined and third-party. The third-party packages are further divided into two subtypes: **local** and **global**.

6.1 Local module

Instructor's explanation
A module installed **inside a project** is a **local module**, and only that project can use it. Suppose you have three Node projects p1, p2 and p3 and you install express inside p1 only. Then only p1 can use it; p2 and p3 must install express again themselves. The installs we did so far (`npm i express`, `npm i cors`) are local installations.

```
npm i package-name
```

6.2 Global module

Instructor's explanation
A **global module** is installed globally in a **shared location**, not inside a specific project, so that **all the projects** on the system can use it. Install it once and every project (ten projects, for example) can utilise it. Global modules are not installed inside a project.

Use the **-g** flag (g stands for global):

```
npm i -g package-name
npm install -g package-name
```

On Windows, global modules are installed in the installation directory of Node.js, that is, under Program Files in the nodejs folder on the C drive.

6.3 Which one should we use? The deciding factor

Instructor's explanation
Rule 1: if a module provides you a **predefined code base**, in the form of functions, properties, classes, interfaces and so on, you **must install it locally**. You cannot install it globally. For example, later we will use functions and classes from the **express** module to simplify server development; there is no other option than a local install.

Instructor's explanation
Rule 2: if a module/package provides a **tool** instead of a code base, install it **globally**. These tools do not give you functions to call in your code. Examples mentioned: tools for testing your application, generating its build, checking formatting and indentation, checking code quality and naming conventions, running a deployment pipeline. A code-quality tool could, for example, report that a given line is missing an indentation, or warn when you concatenate a string with a variable using + instead of the template-literal style that your rules require. Installing such a tool in every project (p1, p2, p3) makes no sense, so install it once globally.

Example given in class of a tool installed globally: **nodemon**.

```
npm i -g nodemon
```

Additional note (not from the lecture)
The instructor named nodemon only as an example of a global tool. In practice, nodemon restarts your Node server automatically when a file changes, which saves the manual Ctrl + C and restart you had to do in class.

6.4 Why is a local module called a dependency?

Instructor's explanation
If you call functions or classes provided by express or another third-party module, your application code is **strictly dependent** on that module: without it, the code cannot run. That is why such a module is a **dependency** of the project and why the key in package.json is called **dependencies**. When someone says 'install one dependency', they mean a package that provides functions or properties to your code.

Global tools are **not** dependencies of any project. Your application works perfectly without them; they only make development better and more organised. So their record is **not** maintained in the dependencies key of package.json. Only local modules are listed there.

Point	Local module	Global module
Installed	Inside one project	In a shared location on the system
Command	`npm i package-name`	`npm i -g package-name`
Who can use it	Only that project (p1, not p2 or p3)	All projects on the system
Provides	Predefined code base: functions, properties, classes, interfaces	A tool: testing, build, code quality, formatting, CLI tools
Listed in dependencies?	Yes, it is a dependency	No, it is not a dependency
Example	express, cors, bcrypt	nodemon
Rule	Must be local if you use its code	Recommended for tools

Student question: does software download node_modules after installation?

Sumeet asked whether, when we install some software, the extra data it downloads is a node_modules folder. The instructor's answer: it depends on the software. The node_modules folder belongs to Node-based applications (back-end server applications and front-end applications that run on Node.js). Desktop applications that depend on other technologies, for example Windows apps and .NET framework apps, rely on their own mechanisms.

7. The ECMAScript import Statement

7.1 Phase 1: the CommonJS way (require)

So far, to use any module we first have to `require` it. This belongs to the **CommonJS convention**.

```
const http = require("http");
const server = http.createServer((request, response) => { /* ... */ });
```

7.2 Phase 2: the ECMAScript way (import)

Instructor's explanation
The ECMAScript convention introduced the **import statement**. It does the same job as require but in a **more readable and better organised** way. JavaScript is highly inspired by Java, which has the import statement, so the same idea was introduced here. Nowadays **every modern application follows the ECMAScript convention**. Some legacy applications are too large to switch to import, so there you must still use require; but for any new application use the import syntax.

Syntax: `import { name1, name2 } from "module-or-path";` Put the functions or properties you want inside the curly braces, then *from*, then the module name (or the path for your own module).

```
// http module: only the function we need
import { createServer } from "http";

// own module (current folder, so ./ is needed)
import { sum, product } from "./mymodule.js";
```

CommonJS (require)	ECMAScript (import)
`const http = require("http");` `http.createServer(...)`	`import { createServer } from "http";` `createServer(...)`
`const m = require("./mymodule.js");` `m.sum(1, 2)`	`import { sum, product } from "./mymodule.js";` `sum(1, 2)`

Notice that you no longer write *moduleReference.functionName*; you call the imported function directly.

7.3 Rewriting index.js with import

```
// index.js
import { createServer } from "http";

const server = createServer((request, response) => {
    response.write("hello world");
    response.end();
});

server.listen(5000);
```

Explanation: `import { createServer } from "http"` brings in just the createServer function, so we call `createServer(...)` directly instead of `http.createServer(...)`. It returns the server reference, which we store in `server`. The callback has request and response, we write 'hello world' and end the response, and `server.listen(5000)` starts the server on port 5000.

Problem: running it with the default configuration

```
node index.js

Output:
Error: Cannot use import statement outside a module
```

Instructor's explanation
Who is the culprit? The **type** property in package.json is set to **commonjs**, which makes Node treat the files in the CommonJS way (require). To use the ECMAScript convention such as the import statement, you have to change the type to **module**. When it is set to module, the type field tells Node that all the JS files in the package are ES modules.

Solution: set type to module

```
// package.json
{
  "name": "my-app",
  "version": "1.0.0",
  "main": "index.js",
  "type": "module"
}

node index.js

Output:
(no error; the server keeps running)
```

Open the browser and go to `127.0.0.1:5000`. The page shows **hello world**.

Common mistake
Using `import` while package.json still says `"type": "commonjs"` gives the error 'Cannot use import statement outside a module'. Change type to module. (Remember you can also choose this while answering the `npm init` prompts.)

8. The export Keyword and Importing Your Own Module

8.1 Phase 1: the CommonJS export (from the last session)

```
// mymodule.js (CommonJS)
exports.sum = function (a, b) {
    return a + b;
};

exports.product = function (a, b) {
    return a * b;
};
```

8.2 Phase 2: the ECMAScript export

Instructor's explanation
The `exports.propertyName = ...` syntax is a complex, confusing syntax. With the latest ECMAScript convention you can directly create a function (or property, or class) and simply put the **export keyword in front of it**. That is all.

```
// mymodule.js (ECMAScript)
export function sum(a, b) {
    return a + b;
}

export function product(a, b) {
    return a * b;
}
```

8.3 Using the module in index.js

Purpose: calculate the sum and product of 10 and 20 using our own module and send the result as the response.

```
// index.js
```

```
import { createServer } from "http";
import { sum, product } from "./mymodule.js";

const server = createServer((request, response) => {
    const sumRs = sum(10, 20);
    const productRs = product(10, 20);

    response.write(`sum of 10 and 20 is ${sumRs} and product of 10 and 20 is ${productRs}`);
    response.end();
});

server.listen(5000);
```

Explanation

- `import { sum, product } from "./mymodule.js"`: the path starts with `./` because the file is in the current folder; the two function names inside the braces must match the exported names.
- `sum(10, 20)` and `product(10, 20)` are called directly, with no module reference and no dot.
- The backtick string (template literal) puts the two results into one message using `${...}`.
- Because we changed the source code, the server has to be **restarted manually**: stop it with Ctrl + C and run `node index.js` again.

```
Output:
sum of 10 and 20 is 30 and product of 10 and 20 is 200
```

Why this output: 10 + 20 = 30 and 10 x 20 = 200; the browser shows the text written by response.write.

Instructor's explanation
So now you have the business of **export and import**, which every JavaScript developer must know: in the module file put `export` before what you want to share, and in the other file use `import { ... } from`.

Task	CommonJS	ECMAScript
Export a function	`exports.sum = function(a, b){...}`	`export function sum(a, b){...}`
Import it	`const m = require("./mymodule.js")`	`import { sum } from "./mymodule.js"`
Call it	`m.sum(2, 5)`	`sum(2, 5)`
package.json type	`"commonjs"` (default)	`"module"`

Additional note (not from the lecture)
In ES modules the file extension in the path (`./mymodule.js`) is required in Node.js; the instructor wrote it with the extension. Also, a file that uses the old `exports.sum = ...` style should be converted when type is module.

9. Revision Material

9.1 Summary of all major topics

- **window.name** is a predefined window property, so a top-level variable called name always has a value in the browser; other undeclared names give undefined.
- **Three module types:** predefined (in Node.js), user-defined (by the developer), third-party (by another organisation, installed manually).
- **npm** (Node Package Manager) is a CLI tool installed with Node.js. `npm i name` installs, `npm uninstall name` removes. It also builds, tests and runs scripts.
- Installing a third-party module creates **node_modules**, **package.json** and **package-lock.json**; later installs only update them.
- **package.json** is mandatory in every Node application: a JSON object with project configuration (name, version, description, author, licence, repository, scripts, main, type, dependencies). Create it with `npm init`.
- Entry point convention: **index.js**.
- Installed modules are recorded under the **dependencies** key so the team knows what is used and so the project can be rebuilt with `npm install`.
- **node_modules** holds the downloaded source code; it is huge (hundreds of MB) so it is never transferred; move the source and run npm install.
- **package-lock.json** maps dependencies to files/folders in node_modules; auto-generated, do not edit.
- **Local** modules (predefined code base) are dependencies; **global** modules (`-g`, tools like nodemon) are not.
- **CommonJS:** require / exports. **ECMAScript:** import / export; needs `"type": "module"` in package.json.

9.2 Important definitions and syntax

Term / Syntax	Meaning
Third-party module	A module developed and maintained by another organisation, installed manually
npm	Node Package Manager, a CLI tool to install and manage packages
`npm i name`	Install a package locally
`npm uninstall name`	Uninstall a package
`npm init`	Create package.json by answering prompts
`npm install` (no name)	Install everything listed in dependencies
`npm i -g name`	Install a package globally
package.json	Project configuration file in JSON format; mandatory in a Node app
package-lock.json	Auto-generated lock file mapping dependencies to node_modules contents
node_modules	Folder containing the downloaded code of the installed modules
dependencies	Key in package.json listing the third-party modules the project needs
Dependency	A module whose functions/classes your code uses, so your code depends on it
index.js	Conventional entry point (starting file) of a Node application
`import { a } from "x"`	ECMAScript way to bring in names from a module
`export function f(){}`	ECMAScript way to share a function
`"type": "module"`	Tells Node to treat files as ES modules

9.3 Key differences between related concepts

Concept A	Concept B	Key difference
Predefined module	Third-party module	Comes with Node.js vs developed by another organisation and installed manually
User-defined module	Third-party module	Created by your own team vs created by another organisation
package.json	package-lock.json	Project configuration and dependency list vs auto-generated internal mapping
package.json	node_modules	Record of what is installed vs the actual downloaded code
Local module	Global module	One project only, provides code (dependency) vs all projects, provides a tool
npm i name	npm install	With a name installs that package; without a name installs all dependencies
require / exports	import / export	CommonJS (older, legacy apps) vs ECMAScript (modern, more readable)
type: commonjs	type: module	Default CommonJS vs ES module syntax enabled

9.4 Common mistakes to avoid

Mistake	Why it fails	Correction
Using import while type is commonjs	Error: Cannot use import statement outside a module	Set type to module in package.json
Installing a module but not having it in dependencies	Missing on other systems, errors at run time	Always keep the entry under dependencies (npm does it for you when you use npm i)
Copying node_modules to another system or Git	Huge size, thousands of files	Copy only source code, then run npm install
Editing package-lock.json by hand	It is internal and auto-generated	Never touch it
Installing a code-providing module globally (like express)	Projects cannot use its functions from a global install	Install it locally with npm i name
Installing a tool locally in every project	Wasteful duplication	Install tools globally with -g
Forgetting to restart the server after changing code	Server still runs the old code	Ctrl + C, then node index.js again
Forgetting ./ for your own module path	Treated as a package name; cannot find module	Use ./mymodule.js
Creating a Node app without package.json	You miss important configuration	Run npm init first

9.5 Interview questions and answers

The instructor said the points on package.json, node_modules and dependencies can be asked in interviews. Items marked (extra) go beyond the lecture.

Question	Answer
What are the types of modules in Node.js?	Predefined, user-defined and third-party; third-party are further local or global.
What is a third-party module?	A module developed and maintained by another organisation, not part of Node.js and not written by you; it must be installed manually.
What is npm?	Node Package Manager, a CLI tool installed with Node.js, used to install/uninstall packages and run builds, tests and scripts.
How do you install and uninstall a package?	npm i name (or npm install name) and npm uninstall name.
What gets created when you install a third-party module?	node_modules folder, package.json and package-lock.json.
What is package.json?	A mandatory JSON configuration file of a Node app: name, version, description, author, licence, repository, scripts, main file, dependencies, and more.
How do you create package.json?	With npm init, which asks for the values and generates the file.
Why is index.js used?	It is the conventional entry point, so developers recognise the starting file.
Why must installed packages be in the dependencies key?	So the team knows what is used and so npm install can reinstall them on another system.
What does node_modules contain?	The downloaded JavaScript source code of the installed third-party modules.
Why do we not transfer node_modules?	It can be hundreds of MB with thousands of folders and lakhs of files; move the source and run npm install instead.
What does npm install do with no package name?	Reads package.json and installs everything under dependencies.

Question	Answer
What is package-lock.json?	An auto-generated file that maps each dependency to its files and folders in node_modules; do not edit it.
Local vs global module?	Local: inside one project, provides a code base, it is a dependency. Global: installed with -g in a shared location, provides a tool, not a dependency.
Which modules should be installed locally?	Any module that gives functions, properties, classes or interfaces your code uses (for example express).
How do you fix 'Cannot use import statement outside a module'?	Set "type": "module" in package.json.
Difference between require and import?	require is CommonJS; import is the modern ECMAScript statement, more readable, used in modern applications.
(extra) Does npm install -g add to dependencies?	No; global installs are outside the project and do not change package.json.

9.6 Practice exercises (solutions not included)

Easy

1. List the three types of modules with one example of each.
2. Which command installs a package? Which removes it?
3. Name three things npm init asks you for.

Medium

1. Create a new folder, run npm init, accept the defaults, and open the generated package.json. Which keys do you see?
2. Install express and cors in that folder. What changes in package.json? What new folder and file appear?
3. Check the size and the number of files in node_modules. Now delete the folder and recreate it with npm install.
4. Explain in your own words why node_modules is not copied to Git.

Challenging

1. Write a module with two exported functions, difference(a, b) and square(a), using the ECMAScript export keyword. Import them in index.js and show the results in the browser on port 5000.
2. Make the same server work with CommonJS (require / exports) and then with ES modules. List every change you had to make, including package.json.
3. Decide for each of these whether to install locally or globally and justify it: express, a code-quality checking tool, a testing tool, cors, nodemon.
4. Install bcrypt as a new package and verify that package.json is updated automatically.

9.7 Final revision checklist

- [ ] I can explain why window.name behaves specially in the browser.
- [ ] I can name the three types of modules and explain third-party modules.
- [ ] I know npm is a CLI tool installed with Node.js, and I know npm i / npm uninstall.
- [ ] I know a folder and two files appear after the first installation.
- [ ] I can explain package.json, what it holds and why every Node application needs it.

- [ ] I can create package.json with npm init and explain the entry point index.js.
- [ ] I can explain the dependencies key and its two purposes (team awareness and rebuild).
- [ ] I can explain what node_modules contains and why it is not transferred.
- [ ] I know how to rebuild a project on a new system with npm install.
- [ ] I know package-lock.json is auto-generated and must not be edited.
- [ ] I can distinguish local and global modules and apply the deciding factor (code base vs tool).
- [ ] I can explain why a local module is called a dependency.
- [ ] I can rewrite require code using import, and set type to module when needed.
- [ ] I can create a module with the export keyword and import it with a ./ path.